

LESSON 03: INDEXING

At first, indexing in MATLAB seems like a trivial topic, and for the most part, the basics will suffice for most *basic* operations. However, once you start trying to vectorize more complicated operations, especially ones that rely on conditions, you'll quickly find yourself limited in what you can calculate.

Subscript Indexing

For the remainder of this lesson, we'll be working with the following:

$$\mathbf{x} = [1, 2, 3, 4, 5, 6, 7, 8] \quad (1)$$

$$\mathbf{A} = \begin{bmatrix} 1 & 5 & 9 & 13 \\ 2 & 6 & 10 & 14 \\ 3 & 7 & 11 & 15 \\ 4 & 8 & 12 & 16 \end{bmatrix} \quad (2)$$

We've already seen that we can access multiple elements of a vector by indexing with another vector:

```
1 x([3, 1, 2])
```

```
1 ans =
```

```
2
3      3      1      2
```

But we can also write to these elements:

```
1 y = x;
```

```
2 y([3, 1, 2]) = -1
```

```
1 y =
```

```
2
3     -1     -1     -1      4      5      6      7      8
```

Matrices behave much the same as this behavior extends to each dimension:

```
1 A([1, 4], [2, 3])
```

```
1 ans =
```

```
2
3      5      9
4      8     12
```

Say we want to select all elements of a dimension; we can use a bare `:` in place of `1:end` to save us some typing:

```
1 A(:, 3)
_____
1 ans =
2
3      9
4     10
5     11
6     12
```

Linear Indexing

Last lesson, we flirted with the idea of linear indexing without getting too much into how it works. Essentially, MATLAB stores its arrays in column-major order, and linear indexing is nothing more than taking advantage of this design decision.

Take for example:

```
1 A([4, end]) % [A(4, 1), A(4, 4)]
_____
1 ans =
2
3      4     16
```

You might find yourself having to convert between subscripts and linear indices depending on what you're trying to achieve, and for that MATLAB has the `sub2ind()` and `ind2sub()` functions.

As the names may suggest, they return return one indexing regime given the other. The only requirement for these functions is the size of the array you're working with:

```
1 [row, col] = ind2sub(size(A), 6)
_____
1 row =
2
3      2
4
5 col =
6
7      2
```

```

1 ind = sub2ind(size(A), row, col)


---


1 ind =
2
3      6

```

Logical Arrays

We've finally reached the point where we can introduce a new datatype: the *logical* datatype. MATLAB logical values operate over $\mathbb{Z}_2$ where 0 denotes `false`, and 1 denotes `true`.

What makes logical arrays so powerful is that they obey the rules of Boolean Algebra and allow for us to extract elements of arrays.

Operator	Purpose
&	Logical AND
&&	Logical AND with short-circuiting
~	Logical NOT
	Logical OR
	Logical OR with short-circuiting
<code>xor</code>	Logical exclusive-OR
<code>all</code>	Determine if all array elements are nonzero or true
<code>any</code>	Determine if any array elements are nonzero

Table 1: Logical Operations

Say we wanted to extract the corners of **A**. We could create a logical array with `logical()` with 1's in the location of the elements we'd like to extract:

```

1 corners = logical([
2             1, 0, 0, 1
3             0, 0, 0, 0
4             0, 0, 0, 0
5             1, 0, 0, 1
6             ]);

```

We can then use this logical array to index into **A**:

```

1 A(corners)
_____
1 ans =
2
3     1
4     4
5    13
6    16

```

Relational Operations

Now that we know about logical values, we can combine this power with relational operators to select elements satisfying a condition.

Operator	Purpose
==	Equality
>=	Greater than or equal to
>	Greater than
<=	Less than or equal to
<	Less than
~=	Inequality
<code>isequal</code>	Array equality
<code>isequaln</code>	Array equality, treating NaN values as equal

Table 2: Relational Operations

For example:

```

1 A < 7
_____
1 ans =
2
3    4x4 logical array
4
5     1     1     0     0
6     1     1     0     0
7     1     0     0     0
8     1     0     0     0

```

Notice how the result of this operation is a logical array, which might tempt you to utilize our previously learned logical operators!

Say we wanted to get all values in the domain (3, 7):

```
1 A < 7 & A > 3
_____
1 ans =
2
3 4x4 logical array
4
5 0 1 0 0
6 0 1 0 0
7 0 0 0 0
8 1 0 0 0
```

As another example, we can use the `find()` function to get the linear index of the element(s) that equal 5:

```
1 find(A == 5)
_____
1 ans =
2
3 5
```

As a final example, let's select all the columns of **A** that sum to a value greater than 30:

```
1 satisfy = sum(A) > 30
2 oops = A(satisfy)
_____
1 satisfy =
2
3 1x4 logical array
4
5 0 0 1 1
6
7 oops =
8
9 3 4
```

Oops, it looks like we didn't get what we wanted! Unfortunately, MATLAB doesn't broadcast when indexing, so we'll need to use `repmat()` to generate the logical matrix we desire and a `reshape()` to get the final output in the correct format.

```
1 gucci = repmat(satisfy, 4, 1)
2 slay = A(gucci)
3 cols = reshape(slay, 4, [])
```

```
1 gucci =
2
3     4x4 logical array
4
5      0     0     1     1
6      0     0     1     1
7      0     0     1     1
8      0     0     1     1
9
10 slay =
11
12      9
13     10
14     11
15     12
16     13
17     14
18     15
19     16
20
21 cols =
22
23      9     13
24     10     14
25     11     15
26     12     16
```

As you can see, these operations are *powerful* but may require you to massage the outputs to get it into the correct format. At first, they may seem unintuitive, but like all things in life, you'll get the hang of it with persistence and patience.